

ระบบจัดการคลังหนังสือส่วนตัว (Personal Book Library)

กำหนดเวลาและการส่งงาน (Deadline & Submission)

กำหนดส่งงาน: ภายใน 7 วันปฏิทิน นับจากวันที่ได้รับโจทย์นี้

เมื่อทำเสร็จ กรุณาส่งงานตามนี้:

- Push โค้ดทั้งหมด (Frontend + Backend) ขึ้น GitHub เป็น repository แล้วส่งลิงก์ repository กลับมา (ถ้าตั้งเป็น private ให้ติดต่อกลับเพื่อเชิญ collaborator) กรุณาส่ง commit history ตามจริง ไม่ squash รวมเป็น commit เดียวก่อนส่ง
- แนบ API collection หรือสคริปต์ทดสอบ endpoint ทั้งหมดที่สร้างไว้ไว้ในโปรเจกต์ ใช้เครื่องมืออะไรก็ได้ เช่น Bruno , Postman, Insomnia, ไฟล์ .http/.rest, หรือสคริปต์ curl เพื่อให้กรรมการตรวจสอบ API ได้สะดวก
- แนบไฟล์ README.md อธิบายวิธีตั้งค่าโปรเจกต์ให้ครบสำหรับกรรมการที่จะตรวจสอบ ได้แก่ stack ที่เลือกใช้ (Backend/Frontend/Database), วิธีติดตั้ง dependency, การตั้งค่า environment variable ที่จำเป็น (เช่น DB connection string, JWT secret), วิธีรัน migration, วิธีรัน Backend และ Frontend, และ username/password สำหรับทดสอบ Login
- แนบไฟล์ REFLECTION.md สั้นๆ (ไม่เกิน 10 บรรทัด) เขียนด้วยคำพูดของตัวเอง อธิบายว่าส่วนไหนของโจทย์ที่ท้าทายที่สุดและทำไม

**หมายเหตุ กรุณาเตรียมความพร้อมในการนำเสนอและตอบคำถามในวันสัมภาษณ์*

ภาพรวมของโปรเจกต์

- เป้าหมาย: สร้างเว็บแอปพลิเคชันสำหรับดูรายชื่อหนังสือ เพิ่ม ค้นหา และลบหนังสือ โดยหนังสือแต่ละเล่มมีความสัมพันธ์กับผู้แต่ง (Author) และหมวดหมู่ (Category) พร้อมระบบยืนยันตัวตน (Authentication) เพื่อทดสอบความเข้าใจการทำงานแบบ MVC, การออกแบบ REST API, หลักการ OOP และการออกแบบฐานข้อมูลเชิงสัมพันธ์ (Relational DB Design)
- เทคโนโลยีที่ใช้ (เลือกได้ตามถนัดไม่ผูกกับ framework ใดโดยเฉพาะ):
Backend เลือกได้ระหว่าง .NET Core (ASP.NET Core Web API) หรือ Node.js (framework อะไรก็ได้)
Frontend เลือกได้ระหว่าง Angular หรือ React.js/Next.js Database เป็น relational database อะไรก็ได้
ทุกจุดในโจทย์ที่มีรายละเอียดเฉพาะ stack จะแบ่งเป็นหัวข้อย่อยกำกับไว้ อ่านเฉพาะฝั่งที่ตรงกับ stack ที่ท่านเลือกข้ามฝั่งที่ไม่เกี่ยวข้องได้เลย

ส่วนที่ 1: Backend สถาปัตยกรรม MVC และ REST API

- แยกชั้นสถาปัตยกรรมแบบ MVC/Layered ให้ชัดเจน อย่างน้อย 3 ชั้น: Controller, Service, Repository/Model ห้ามรวมทุกอย่างไว้ในไฟล์เดียว
 - ถ้าเลือก .NET Core: ใช้ Controller class ที่สืบทอดจาก ControllerBase, แยก Service เป็น interface + implementation แล้ว inject ผ่าน Dependency Injection ของ ASP.NET Core, เข้าถึงข้อมูลผ่าน EF Core DbContext ในชั้น Repository
 - ถ้าเลือก Node.js: แยกไฟล์ตามหน้าที่ (เช่น controllers/, services/, repositories/ หรือ models/) เขียน Service/Repository เป็น class ที่มี method ชัดเจน ไม่ใช่ route handler function เดี่ยวๆ ที่คุยกับ DB ตรงๆ

- ตั้งค่า Server และเชื่อมต่อดูฐานข้อมูลจริง เช่น อาจ log ข้อความตอน server พร้อมใช้งานทำนอง 'Book library server is up and ready to roll'
- API Endpoints หลัก (resource-based, ใช้ HTTP verb และ status code ให้ถูกต้องตาม REST convention):
 - GET /api/books: ส่งรายการหนังสือทั้งหมด รองรับ query filter ตาม categoryId และ/หรือ authorId (เช่น /api/books?categoryId=2)
 - GET /api/books/:id: ส่งข้อมูลหนังสือเล่มเดียวตาม id ตอบ 404 ถ้าไม่พบ
 - POST /api/books: สร้างหนังสือใหม่ (ต้องอ้างอิง categoryId และ authorId(s) ที่มีอยู่จริง) ตอบ 201 พร้อมข้อมูลที่สร้าง
 - DELETE /api/books/:id: ลบหนังสือตาม id ตอบ 204 เมื่อสำเร็จ 404 ถ้าไม่พบ
 - GET /api/authors และ GET /api/categories: ส่งรายชื่อผู้แต่ง/หมวดหมู่ทั้งหมด สำหรับใช้ทำ dropdown ฝั่ง frontend
- CORS: ตั้งค่า Middleware/Policy CORS ให้เรียบร้อยเพื่อให้ Frontend เรียกใช้งาน API ข้ามพอร์ตได้

ส่วนที่ 2: การออกแบบฐานข้อมูล (Database Design)

- ออกแบบ schema เอง อย่างน้อย 3 ตารางที่มีความสัมพันธ์กันจริงในระดับฐานข้อมูล
 - Book — ข้อมูลหนังสือ
 - Author — ข้อมูลผู้แต่ง
 - Category — หมวดหมู่หนังสือ
- สร้าง schema ผ่าน migration หรือสคริปต์ที่ทำซ้ำได้
- แบน ER diagram หรือไฟล์ schema (เช่น schema.prisma, migration file, หรือรูปภาพ diagram) ไว้ในโปรเจกต์เป็นหลักฐานการออกแบบ

ส่วนที่ 3: Frontend — เรียกใช้ REST API และออกแบบ Component

- การเชื่อมต่อ API: เรียก endpoint ที่สร้างไว้ใน Backend ทั้งหมด (list, create, delete, filter ตาม category/author)
 - ถ้าเลือก Angular: แยก HttpClient service (Injectable) เป็นคนละไฟล์จาก component โดยเด็ดขาด component เรียกผ่าน service เท่านั้น ไม่ยิง HttpClient ตรงๆ ใน component
 - ถ้าเลือก React: แยกชั้นเรียก API (เช่น module/class/custom hook แยก) ออกจากไฟล์ component ไม่ผสม fetch/axios เข้าไปกลาง JSX โดยตรง
- การจัดการ State: เก็บสถานะรายการหนังสือ ข้อมูลฟอร์มที่กำลังกรอก สถานะ Loading และ Error ให้ครบ พร้อมแสดงผลบน UI ตามสถานะ
- ดึงข้อมูลเริ่มต้นทันทีที่เปิดหน้าเว็บ และอัปเดต UI ทันทีเมื่อมีการเพิ่ม/ลบหนังสือสำเร็จ โดยไม่ต้อง refresh หน้าเว็บเอง
- ฟังก์ชันกรองรายการหนังสือตาม Category และ/หรือ Author บนหน้าจอ

ส่วนที่ 4: ระบบยืนยันตัวตน Backend (Authentication)

- ระบบ Login: สร้าง Endpoint POST /api/login สำหรับรับและตรวจสอบ Username กับ Password
- การสร้าง Token: สร้าง token-based auth (JWT) พร้อมกำหนดวันหมดอายุ แล้วส่งกลับไปให้ Frontend
 - ถ้าเลือก .NET Core: ใช้ System.IdentityModel.Tokens.Jwt (หรือไลบรารีอื่นๆ) สร้าง JWT และตั้งค่า JWT Bearer Authentication middleware ของ ASP.NET Core
 - ถ้าเลือก Node.js: ใช้ไลบรารี jsonwebtoken (หรือไลบรารีอื่นๆ) สร้าง token

- Middleware/Filter ป้องกัน API: กรณีไม่มี token หรือ token ไม่ถูกต้อง ต้องตอบกลับ 401
 - ถ้าเลือก .NET Core: ใช้ [Authorize] attribute ครอบ action/controller ที่ต้องป้องกัน ร่วมกับ JWT Bearer middleware ที่ตั้งค่าไว้
 - ถ้าเลือก Node.js: เขียน middleware ตรวจสอบ Token จาก Header (Authorization: Bearer <token>) เอง แล้วนำไปครอบเฉพาะ route ที่ต้องป้องกัน
 - นำการป้องกันนี้ไปครอบ Endpoint POST และ DELETE ของ /api/books เพื่อบังคับว่าต้องล็อกอินก่อนจึงจะจัดการหนังสือได้

ส่วนที่ 5: การจัดการ Token ฝั่ง Frontend

- หน้าจอ Login: สร้างหน้า/Component Login เพื่อรับข้อมูลและยิง API ไปยัง /api/login
- การจัดเก็บและการแนบ Token: บันทึก Token ที่ได้รับลงในฝั่ง client
 - ถ้าเลือก Angular: ตั้ง Token แนบ Header อัตโนมัติผ่าน HttpInterceptor
 - ถ้าเลือก React: ตั้ง Token แนบ Header อัตโนมัติผ่าน axios interceptor หรือ wrapper function รอบ fetch
- Auth Guard: ตรวจสอบ Token ตั้งแต่เปิดเว็บ หากไม่มีให้ Redirect ผู้ใช้กลับไปหน้าจอ Login ทันที เช่น อาจแสดงข้อความก่อน redirect ทำนอง 'กรุณาเข้าสู่ระบบก่อนใช้งาน'
 - ถ้าเลือก Angular: ใช้ Route Guard (CanActivate)
 - ถ้าเลือก React: ใช้ Protected Route component ห่อ route ที่ต้อง login
- จัดการ Token หมดอายุ: หากเรียก API แล้ว Backend ตอบกลับมาเป็น Error 401 ให้ลบ Token เก่าทิ้ง และพาผู้ใช้ออกจากระบบ

โบนัส (Optional): Deploy จริง + Docker สำหรับคนที่มีเวลา/ความสามารถเพิ่มเติม

- ไม่บังคับ ไม่มีผลต่อการผ่านเกณฑ์หลัก เป็นคำแนะนำพิเศษเพิ่มเติมสำหรับคนที่อยากท้าทายตัวเองเพิ่ม
- Dockerize: เขียน Dockerfile สำหรับทั้ง Backend และ Frontend พร้อม docker-compose.yml ให้รันคู่กันได้ด้วยคำสั่งเดียว เช่น docker compose up
- Deploy ขึ้นจริง: นำแอปขึ้น hosting ฟรีที่ไหนก็ได้ (เช่น Render, Railway, Fly.io, Vercel, Azure Free Tier) แล้วแนบลิงก์ที่ใช้งานได้จริงไว้ใน README.md

เกณฑ์การประเมินและทดสอบตัวเอง (Checklist)

- หน้าเว็บโหลดรายชื่อหนังสือจาก Backend มาแสดงผลได้จริง มีสถานะ Loading ระหว่างรอข้อมูล และแสดง error เมื่อเรียก API ไม่สำเร็จ
- เพิ่ม/ลบหนังสือผ่านหน้าเว็บได้จริง ข้อมูลบนหน้าจออัปเดตทันทีหลังบันทึกสำเร็จ โดยไม่ต้อง refresh เอง
- หากยิง API เพิ่ม/ลบหนังสือผ่าน API client (เช่น Postman) โดยไม่มี Token ระบบ Backend ต้องปฏิเสธ (Error 401)
- เมื่อเข้าเว็บครั้งแรกโดยยังไม่ล็อกอิน ระบบบังคับให้ไปหน้า Login ได้อย่างถูกต้อง
- หลังล็อกอินสำเร็จ Token ถูกเก็บไว้ฝั่ง client และสามารถเพิ่ม/ลบหนังสือได้ตามปกติ
- หากลบ Token ออกจากที่จัดเก็บด้วยตัวเอง ระบบต้องพากลับไปหน้า Login เมื่อพยายามใช้งาน